Natalie Whalen BSAN 360 Project

Lab 4

1. Download and read your data into a data frame. Check that the process does not produce errors and make any necessary adjustments to your read command until it works.

In [354…

```
import pandas as pd
fastfood = pd.read_csv('fastfood.csv')
fastfood
```

Out[354…

	restaurant	item	calories	cal_fat	total_fat	sat_fat	trans_fat	cholester
0	Mcdonalds	Artisan Grilled Chicken Sandwich	380	60	7	2.0	0.0	9
1	Mcdonalds	Single Bacon Smokehouse Burger	840	410	45	17.0	1.5	13
2	Mcdonalds	Double Bacon Smokehouse Burger	1130	600	67	27.0	3.0	22
3	Mcdonalds	Grilled Bacon Smokehouse Chicken Sandwich	750	280	31	10.0	0.5	15
4	Mcdonalds	Crispy Bacon Smokehouse Chicken Sandwich	920	410	45	12.0	0.5	12
…	…	…	…	…	…	…	…	
510	Taco Bell	Spicy Triple Double Crunchwrap	780	340	38	10.0	0.5	5
511	Taco Bell	Express Taco Salad w/ Chips	580	260	29	9.0	1.0	6
512	Taco Bell	Fiesta Taco Salad-Beef	780	380	42	10.0	1.0	6
513	Taco Bell	Fiesta Taco Salad-Chicken	720	320	35	7.0	0.0	7
514	Taco Bell	Fiesta Taco Salad-Steak	720	320	36	8.0	1.0	5

515 rows × 17 columns

2. How many rows and columns does your data frame have? Check that this is the correct size.

In [355…
```
515 * 17
```

Out[355… 8755

3. Check the first and last few rows of data. Does everything look correct?

Yes

4. Make sure that column and row labels got processed correctly. Re-label columns and rows as needed!

In [356…

```
fastfood
```

Out[356...

	restaurant	item	calories	cal_fat	total_fat	sat_fat	trans_fat	cholester
0	Mcdonalds	Artisan Grilled Chicken Sandwich	380	60	7	2.0	0.0	9
1	Mcdonalds	Single Bacon Smokehouse Burger	840	410	45	17.0	1.5	13
2	Mcdonalds	Double Bacon Smokehouse Burger	1130	600	67	27.0	3.0	22
3	Mcdonalds	Grilled Bacon Smokehouse Chicken Sandwich	750	280	31	10.0	0.5	15
4	Mcdonalds	Crispy Bacon Smokehouse Chicken Sandwich	920	410	45	12.0	0.5	12
...	...	...	...	...	...	...	...	
510	Taco Bell	Spicy Triple Double Crunchwrap	780	340	38	10.0	0.5	5
511	Taco Bell	Express Taco Salad w/ Chips	580	260	29	9.0	1.0	6
512	Taco Bell	Fiesta Taco Salad-Beef	780	380	42	10.0	1.0	6
513	Taco Bell	Fiesta Taco Salad-Chicken	720	320	35	7.0	0.0	7
514	Taco Bell	Fiesta Taco Salad-Steak	720	320	36	8.0	1.0	5

515 rows × 17 columns

5. Look at the descriptive statistics and perform the following checks and any necessary corrections for each column:

a. What information is provided in this column?

b. What type of data is given in this column? The types we've seen most are Numeric (integer or real number), Character Strings, Boolean, Datetime. If the data type is not correct, correct it as best you can. (A full resetting of data types may require you to do some handling of missing values, which we will cover on Wednesday, so no worries if you can't take care of everything.)

c. What is the range of values in this column? Check the min and max values to make sure that they are reasonable.

d. Are there any missing values in this column? If so, how do you plan to handle them? The above steps will help you develop a plan to clean your data and create a data dictionary (also called a schema) for your project. A data dictionary will be useful for you to communicate your work to others

a

restaurant (oject): The name of the restaurant chain or brand offering the food item (e.g., McDonald's, Burger King, Wendy's). Identifies the source of the menu item.

calories (int): The total energy content of the food item, measured in kilocalories (kcal). Represents the amount of energy the body would obtain by consuming the item.

cal_fat (int): The number of calories in the item that come specifically from fat. Useful for understanding what proportion of total calories are fat-derived.

total_fat (int): The total amount of fat in grams contained in the item, including saturated and unsaturated fats. A measure of overall fat content.

sat_fat (float): The amount of saturated fat (grams) in the item — the type of fat associated with increased cholesterol levels when consumed in excess.

trans_fat (float): The amount of trans fat (grams) in the item — often found in processed foods; associated with negative cardiovascular effects.

cholesterol (int): The cholesterol content of the item in milligrams (mg). A dietary measure important for assessing heart-health risk.

sodium (int): The sodium (salt) content in milligrams (mg) per serving. High sodium values can indicate high salt intake risk.

total_carb (int): The total carbohydrates (grams) in the food item, including sugars, starches, and dietary fiber. Key nutrient for energy supply.

fiber (float): The dietary fiber content (grams) — the portion of carbohydrates that cannot be digested; beneficial for digestion and heart health.

sugar (int): The total sugar content (grams) in the item, including both natural and added sugars. Important for assessing added sugar intake.

protein (float): The protein content (grams) per serving. Represents the primary source of amino acids for body tissue maintenance and repair.

vit_a (float): The Vitamin A content expressed as a percentage of the recommended daily value (% DV). Supports vision and immune function.

vit_c (float): The Vitamin C content (% DV). Important antioxidant that supports immune system health and tissue repair.

calcium (float): The Calcium content (% DV). Important mineral for bone and teeth health.

salad (object): Indicates whether the menu item is classified as a salad (e.g., “Yes”/“No” or name of salad). Used for category filtering or grouping.

In [357…
```
#b
fastfood.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 515 entries, 0 to 514
Data columns (total 17 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   restaurant   515 non-null    object 
 1   item         515 non-null    object 
 2   calories     515 non-null    int64  
 3   cal_fat      515 non-null    int64  
 4   total_fat    515 non-null    int64  
 5   sat_fat      515 non-null    float64
 6   trans_fat    515 non-null    float64
 7   cholesterol  515 non-null    int64  
 8   sodium       515 non-null    int64  
 9   total_carb   515 non-null    int64  
 10  fiber        503 non-null    float64
 11  sugar        515 non-null    int64  
 12  protein      514 non-null    float64
 13  vit_a        301 non-null    float64
 14  vit_c        305 non-null    float64
 15  calcium      305 non-null    float64
 16  salad        515 non-null    object 
dtypes: float64(7), int64(7), object(3)
memory usage: 68.5+ KB
```

In [358…
```
#c
fastfood.describe()
```

Out [358…

	calories	cal_fat	total_fat	sat_fat	trans_fat	cholesterol
count	515.000000	515.000000	515.000000	515.000000	515.000000	515.000000
mean	530.912621	238.813592	26.590291	8.153398	0.465049	72.456311
std	282.436147	166.407510	18.411876	6.418811	0.839644	63.160406
min	20.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	330.000000	120.000000	14.000000	4.000000	0.000000	35.000000
50%	490.000000	210.000000	23.000000	7.000000	0.000000	60.000000
75%	690.000000	310.000000	35.000000	11.000000	1.000000	95.000000
max	2430.000000	1270.000000	141.000000	47.000000	8.000000	805.000000

d

Yes, in columns fiber, protein, vit_a, vit_c, calcium Since I am only using columns with missing vit_a, vit_c, and calcium, I will not be using those columns to answer my research question.

6. Next, we will focus on the project questions and figure out a roadmap for your project. Restate your project questions from your assignment here, and for each question, determine which columns of data you will need to answer it.

Across McDonalds, Sonic, and Dairy Queen what restaurant has the best health scores across classic menu items using total fat, protein, fiber, calories, and sodium.

7. Create a new data frame that contains all the columns you'll need for your project. (If your computer's memory allows, create individual data frames for each of your research questions.)

In [359…

```
fastfood1 = fastfood.loc[[6, 96, 267, 34, 111, 291, 7, 109, 296]]
fastfood1
```

Out [359…

	restaurant	item	calories	cal_fat	total_fat	sat_fat	trans_fat	choleste
6	Mcdonalds	Cheeseburger	300	100	12	5.0	0.5	
96	Sonic	Sonic Cheeseburger W/ Ketchup	720	380	43	17.0	2.0	
267	Dairy Queen	Original Cheeseburger	400	160	18	9.0	1.0	
34	Mcdonalds	3 piece Buttermilk Crispy Chicken Tenders	370	190	21	3.5	0.0	
111	Sonic	3 Piece Crispy Chicken Tender Dinner	280	130	14	2.5	0.0	
291	Dairy Queen	3 chicken strips Chicken Strips	350	180	20	3.0	0.0	
7	Mcdonalds	Classic Chicken Sandwich	510	210	24	4.0	0.0	
109	Sonic	Crispy Chicken Sandwich	570	300	33	5.0	0.0	
296	Dairy Queen	Crispy Chicken Sandwich	600	270	30	5.0	0.0	

In [360…

```
fastfood2 = fastfood1.drop(columns=["vit_a", "vit_c", "calcium", "salad", "c
fastfood2
```

Out [360…

	restaurant	item	calories	total_fat	sodium	fiber	protein
6	Mcdonalds	Cheeseburger	300	12	680	2.0	15.0
96	Sonic	Sonic Cheeseburger W/ Ketchup	720	43	1190	2.0	35.0
267	Dairy Queen	Original Cheeseburger	400	18	930	1.0	19.0
34	Mcdonalds	3 piece Buttermilk Crispy Chicken Tenders	370	21	910	0.0	28.0
111	Sonic	3 Piece Crispy Chicken Tender Dinner	280	14	800	0.0	22.0
291	Dairy Queen	3 chicken strips Chicken Strips	350	20	960	10.0	22.0
7	Mcdonalds	Classic Chicken Sandwich	510	24	1040	3.0	25.0
109	Sonic	Crispy Chicken Sandwich	570	33	1060	4.0	23.0
296	Dairy Queen	Crispy Chicken Sandwich	600	30	1250	7.0	24.0

8. Going back to your research questions themselves, determine what type of statistical analyses you'll need to answer each one. Some examples could be ANOVA to determine if there are differences among groups, regression for prediction, time series analysis for forecasting, classification or clustering methods.

I would use a Two-Way ANOVA (Factorial ANOVA) to compare mean health scores calculated from total fat, protein, fiber, calories, and sodium across restaurant (McDonald's, Sonic, Dairy Queen) and menu item type (cheeseburger, chicken sandwich, chicken tenders), while also testing for an interaction between them.

Project 2

In [361…
```
fastfood2.loc[6, "item"] = "McDonalds Cheeseburger"
fastfood2.loc[96, "item"] = "Sonic Cheeseburger"
fastfood2.loc[267, "item"] = "DQ Cheeseburger"
fastfood2.loc[34, "item"] = "McDonalds 3-Tenders"
fastfood2.loc[111, "item"] = "Sonic 3-Tenders"
fastfood2.loc[291, "item"] = "DQ 3-Tenders"
fastfood2.loc[7, "item"] = "McDonalds Chicken Sandwich"
fastfood2.loc[109, "item"] = "Sonic Chicken Sandwich"
fastfood2.loc[296, "item"] = "DQ Chicken Sandwich"
```

In [362…
```
df = fastfood2.set_index('item')
df
```

Out[362…

item	restaurant	calories	total_fat	sodium	fiber	protein
McDonalds Cheeseburger	Mcdonalds	300	12	680	2.0	15.0
Sonic Cheeseburger	Sonic	720	43	1190	2.0	35.0
DQ Cheeseburger	Dairy Queen	400	18	930	1.0	19.0
McDonalds 3-Tenders	Mcdonalds	370	21	910	0.0	28.0
Sonic 3-Tenders	Sonic	280	14	800	0.0	22.0
DQ 3-Tenders	Dairy Queen	350	20	960	10.0	22.0
McDonalds Chicken Sandwich	Mcdonalds	510	24	1040	3.0	25.0
Sonic Chicken Sandwich	Sonic	570	33	1060	4.0	23.0
DQ Chicken Sandwich	Dairy Queen	600	30	1250	7.0	24.0

In [363…
```
df.dtypes
```

Out[363…
```
restaurant     object
calories        int64
total_fat       int64
sodium          int64
fiber         float64
protein       float64
dtype: object
```

In [364…
```
df["restaurant"] = df["restaurant"].astype("category")
df.dtypes
```

Out[364…
```
restaurant    category
calories         int64
total_fat        int64
sodium           int64
fiber          float64
protein        float64
dtype: object
```

In [365…
```
df['health_score'] = (df['protein'] + df['fiber']) / (
    (df['calories'] / 100) + (df['total_fat'] / 10) + (df['sodium'] / 1000))
df
```

Out [365…

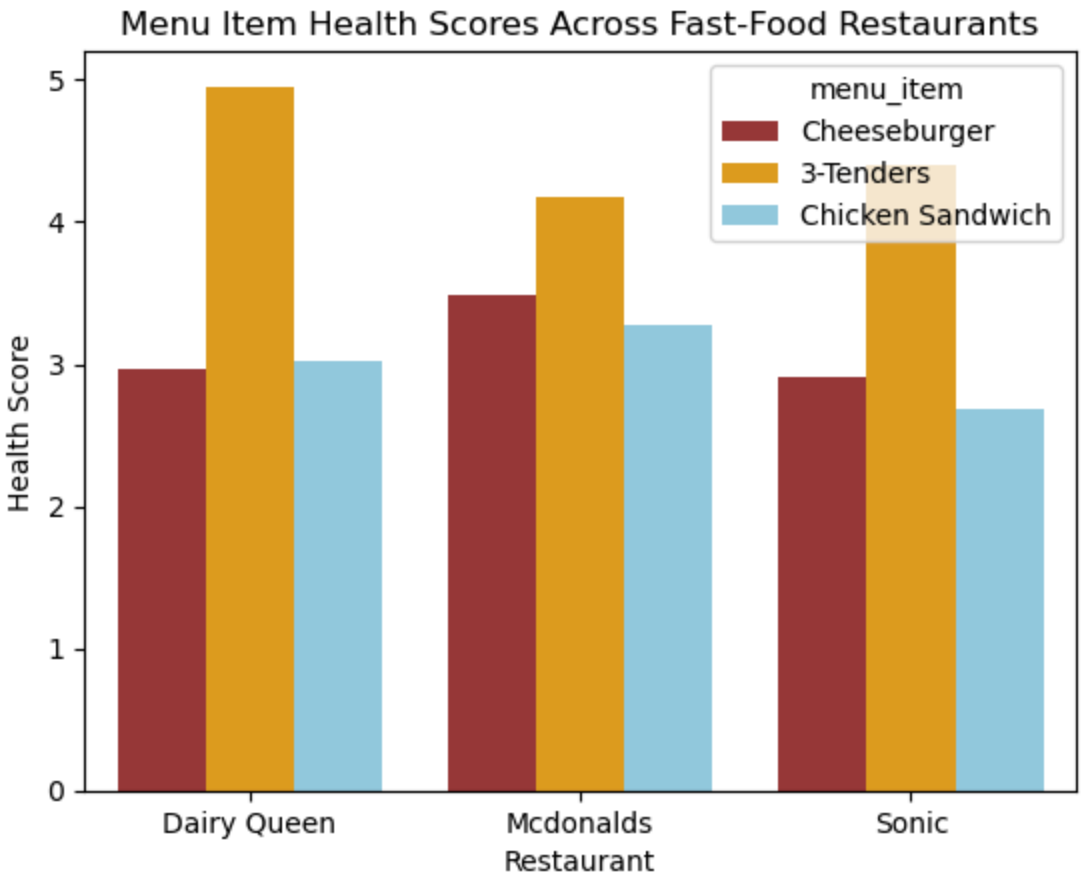
item	restaurant	calories	total_fat	sodium	fiber	protein	health_score
McDonalds Cheeseburger	Mcdonalds	300	12	680	2.0	15.0	3.483607
Sonic Cheeseburger	Sonic	720	43	1190	2.0	35.0	2.915682
DQ Cheeseburger	Dairy Queen	400	18	930	1.0	19.0	2.971768
McDonalds 3-Tenders	Mcdonalds	370	21	910	0.0	28.0	4.172876
Sonic 3-Tenders	Sonic	280	14	800	0.0	22.0	4.400000
DQ 3-Tenders	Dairy Queen	350	20	960	10.0	22.0	4.953560
McDonalds Chicken Sandwich	Mcdonalds	510	24	1040	3.0	25.0	3.278689
Sonic Chicken Sandwich	Sonic	570	33	1060	4.0	23.0	2.683897
DQ Chicken Sandwich	Dairy Queen	600	30	1250	7.0	24.0	3.024390

Project 4

In [366…
```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
```

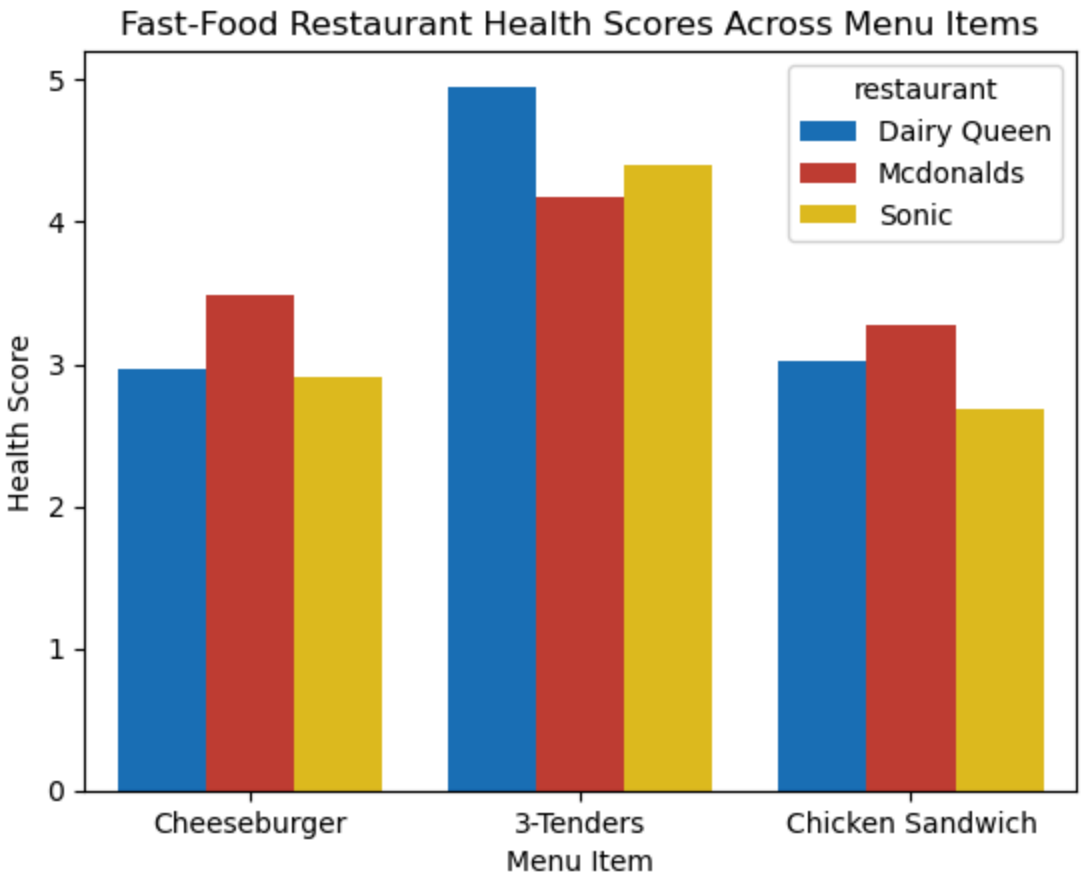
In [367…
```
df = df.reset_index()
df['menu_item'] = df['item'].str.extract(r'(Cheeseburger|3-Tenders|Chicken S
```

In [368…
```
sns.barplot(df,
    x='restaurant',
    y='health_score',
    hue='menu_item',
    palette=['brown', 'orange', 'skyblue']  )
plt.title('Menu Item Health Scores Across Fast-Food Restaurants')
plt.xlabel('Restaurant')
plt.ylabel('Health Score')
plt.show()
```



In [369…

```
sns.barplot(df,
    x='menu_item',
    y='health_score',
    hue='restaurant',
    palette=['#0072CE', '#DA291C', '#FFD100']  )
plt.title('Fast-Food Restaurant Health Scores Across Menu Items')
plt.xlabel('Menu Item')
plt.ylabel('Health Score')
plt.show()
```



In [370…

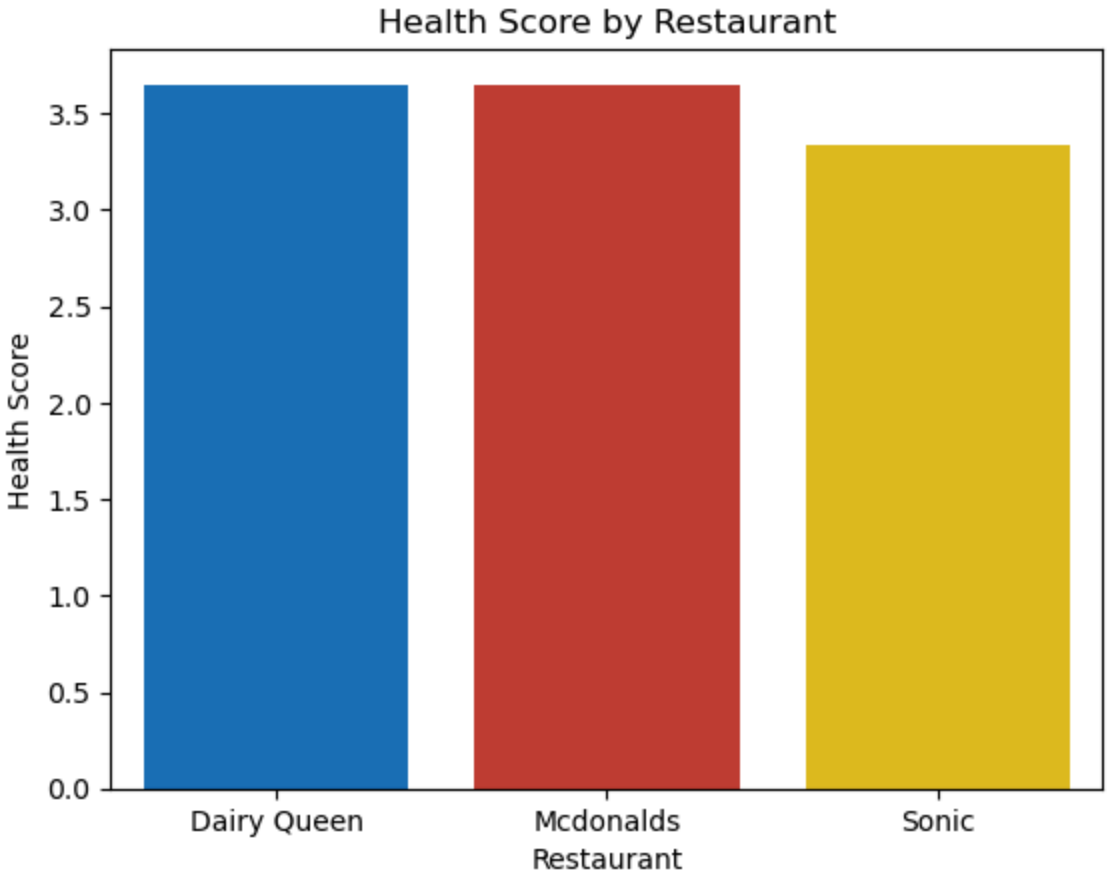
```
hs_mean = df.groupby("restaurant")["health_score"].mean().reset_index()
print (hs_mean)

sns.barplot(
    data=hs_mean,
    x='restaurant',
    y='health_score',
    hue='restaurant',
    palette=['#0072CE', '#DA291C', '#FFD100']
)

plt.title('Health Score by Restaurant')
plt.xlabel('Restaurant')
plt.ylabel('Health Score')
plt.show()
```

```
    restaurant  health_score
0  Dairy Queen      3.649906
1    Mcdonalds      3.645057
2        Sonic      3.333193
```

```
/var/folders/7t/k84809jn0ms4mxh05z5blvmh0000gn/T/ipykernel_17742/1660456985.
py:1: FutureWarning: The default of observed=False is deprecated and will be
changed to True in a future version of pandas. Pass observed=False to retain
current behavior or observed=True to adopt the future default and silence th
is warning.
  hs_mean = df.groupby("restaurant")["health_score"].mean().reset_index()
```



In [371…

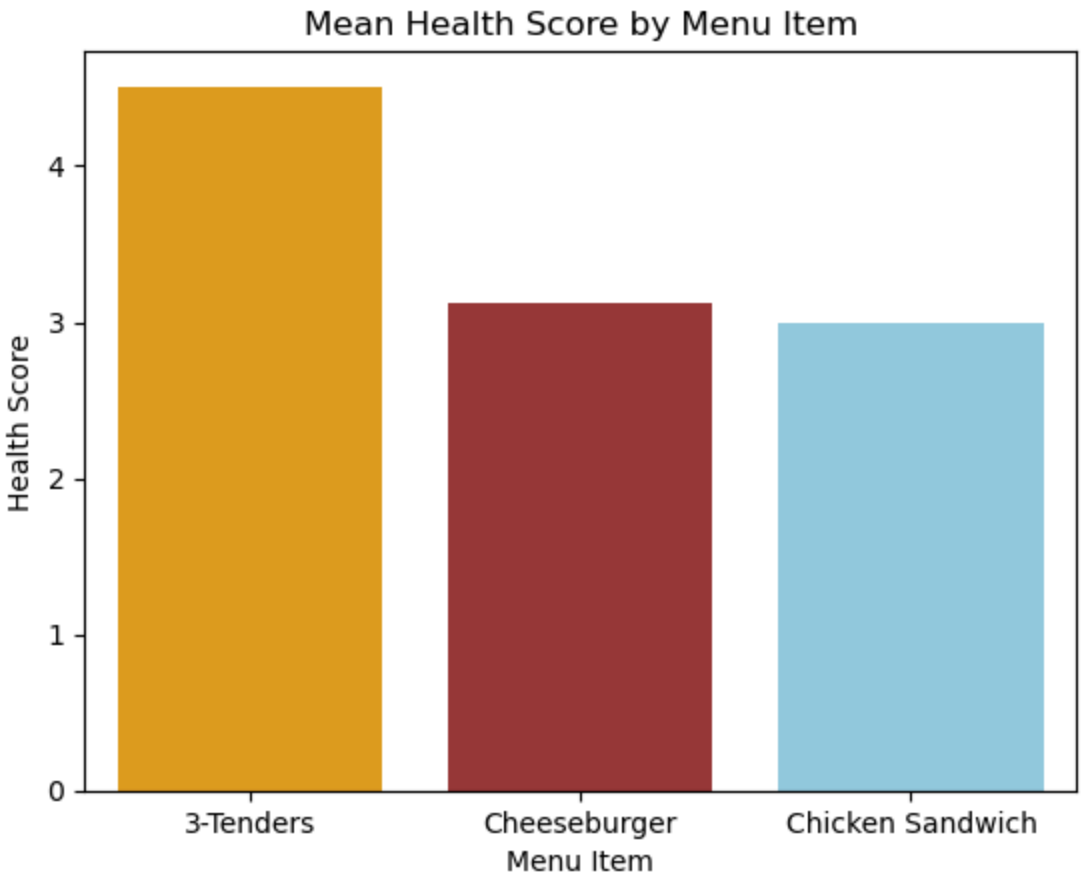
```
item_mean = df.groupby("menu_item")["health_score"].mean().reset_index()
print(item_mean)

sns.barplot(
    data=item_mean,
    x='menu_item',
    y='health_score',
    hue='menu_item',
    palette=['orange', 'brown', 'skyblue']  )


plt.title('Mean Health Score by Menu Item')
plt.xlabel('Menu Item')
plt.ylabel('Health Score')

plt.show()
```

```
          menu_item  health_score
0         3-Tenders      4.508812
1      Cheeseburger      3.123685
2  Chicken Sandwich      2.995658
```



Project 5

In [372…
```
df2= df.groupby(['restaurant','menu_item'])['health_score'].mean()
df2
```

```
/var/folders/7t/k84809jn0ms4mxh05z5blvmh0000gn/T/ipykernel_17742/2218008898.
py:1: FutureWarning: The default of observed=False is deprecated and will be
changed to True in a future version of pandas. Pass observed=False to retain
current behavior or observed=True to adopt the future default and silence th
is warning.
  df2= df.groupby(['restaurant','menu_item'])['health_score'].mean()
```

Out[372…
```
restaurant   menu_item
Dairy Queen  3-Tenders           4.953560
             Cheeseburger        2.971768
             Chicken Sandwich    3.024390
Mcdonalds    3-Tenders           4.172876
             Cheeseburger        3.483607
             Chicken Sandwich    3.278689
Sonic        3-Tenders           4.400000
             Cheeseburger        2.915682
             Chicken Sandwich    2.683897
Name: health_score, dtype: float64
```

In [382…
```
pivot = df.pivot_table(
    index='restaurant',
    columns='menu_item',
```

```
    values='health_score',)
pivot
```

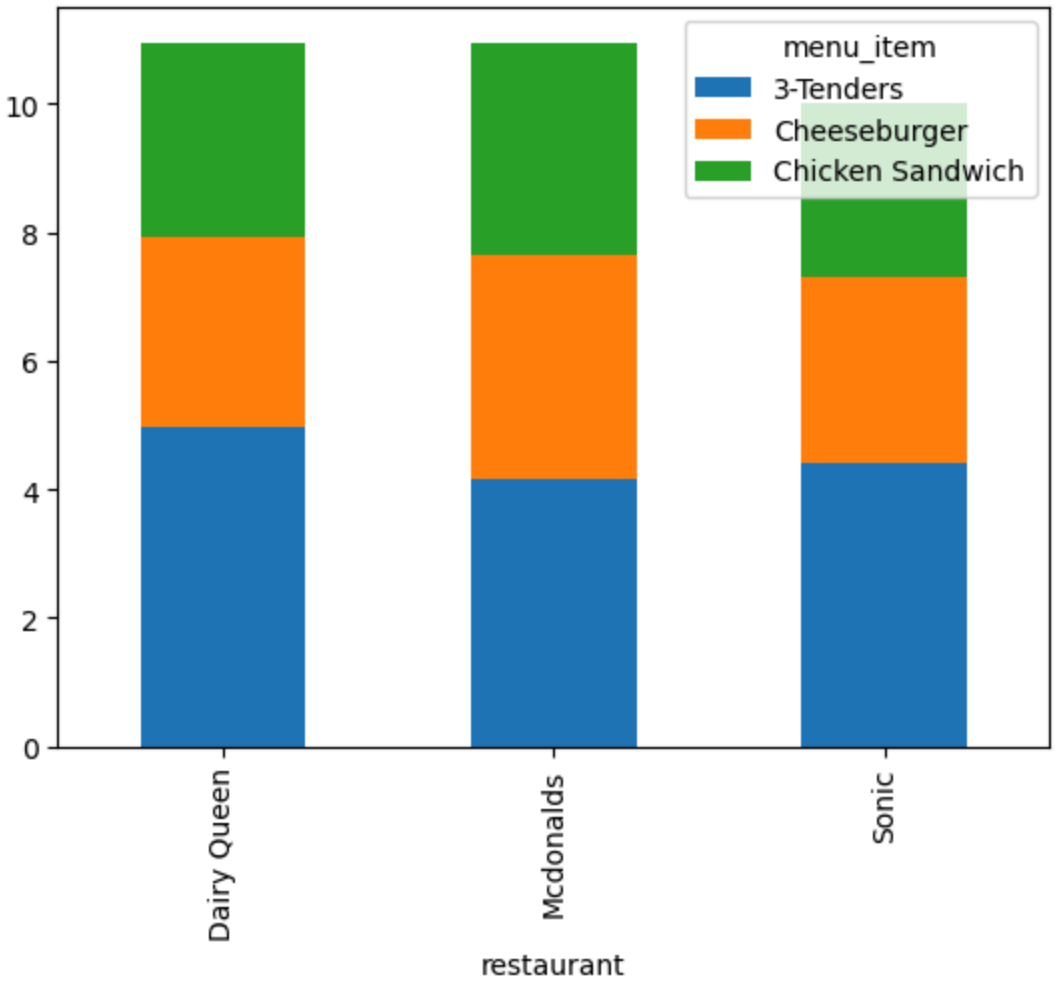
```
/var/folders/7t/k84809jn0ms4mxh05z5blvmh0000gn/T/ipykernel_17742/3787412661.
py:1: FutureWarning: The default value of observed=False is deprecated and w
ill change to observed=True in a future version of pandas. Specify observed=
False to silence this warning and retain the current behavior
  pivot = df.pivot_table(
```

Out[382…

menu_item	3-Tenders	Cheeseburger	Chicken Sandwich
restaurant			
Dairy Queen	4.953560	2.971768	3.024390
Mcdonalds	4.172876	3.483607	3.278689
Sonic	4.400000	2.915682	2.683897

In [374…

```
pivot.plot(kind="bar", stacked=True)
plt.show()
```



Project 6

In [386…

```
from scipy.stats import f_oneway
```

```
dq = df[df['restaurant']=='Dairy Queen']['health_score']
mc = df[df['restaurant']=='Mcdonalds']['health_score']
sonic = df[df['restaurant']=='Sonic']['health_score']

F, p = f_oneway(dq, mc, sonic)
print(F)
print(p)
```

```
0.12549351370084813
0.8843169681935185
```

In [389…

```
from scipy.stats import f_oneway

tenders = df[df['menu_item']=='3-Tenders']['health_score']
burger = df[df['menu_item']=='Cheeseburger']['health_score']
chicken = df[df['menu_item']=='Chicken Sandwich']['health_score']

F, p = f_oneway(tenders, burger, chicken)
print(F)
print(p)
```

```
18.196192490949816
0.0028352424301180544
```

In [388…

```
import statsmodels.formula.api as smf

model = smf.ols("health_score ~ calories + total_fat + sodium + fiber + prot
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
==
Dep. Variable:           health_score   R-squared:                       0.9
86
Model:                            OLS   Adj. R-squared:                  0.9
64
Method:                 Least Squares   F-statistic:                     43.
79
Date:                Wed, 03 Dec 2025   Prob (F-statistic):           0.005
27
Time:                        22:26:06   Log-Likelihood:                 9.31
33
No. Observations:                   9   AIC:                            -6.6
27
Df Residuals:                       3   BIC:                            -5.4
43
Df Model:                           5
Covariance Type:            nonrobust
==============================================================================
==
                 coef    std err          t      P>|t|      [0.025      0.97
5]
------------------------------------------------------------------------------
--
Intercept      3.6229      0.457      7.935      0.004       2.170       5.0
76
calories      -0.0068      0.002     -3.121      0.052      -0.014       0.0
00
total_fat     -0.0059      0.031     -0.188      0.863      -0.106       0.0
94
sodium        -0.0004      0.001     -0.401      0.715      -0.004       0.0
03
fiber          0.1341      0.026      5.197      0.014       0.052       0.2
16
protein        0.1313      0.023      5.715      0.011       0.058       0.2
04
==============================================================================
==
Omnibus:                        3.217   Durbin-Watson:                   2.5
85
Prob(Omnibus):                  0.200   Jarque-Bera (JB):                0.8
07
Skew:                           0.707   Prob(JB):                        0.6
68
Kurtosis:                       3.390   Cond. No.                     1.01e+
04
==============================================================================
==

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is corre
ctly specified.
[2] The condition number is large, 1.01e+04. This might indicate that there
are
strong multicollinearity or other numerical problems.
```

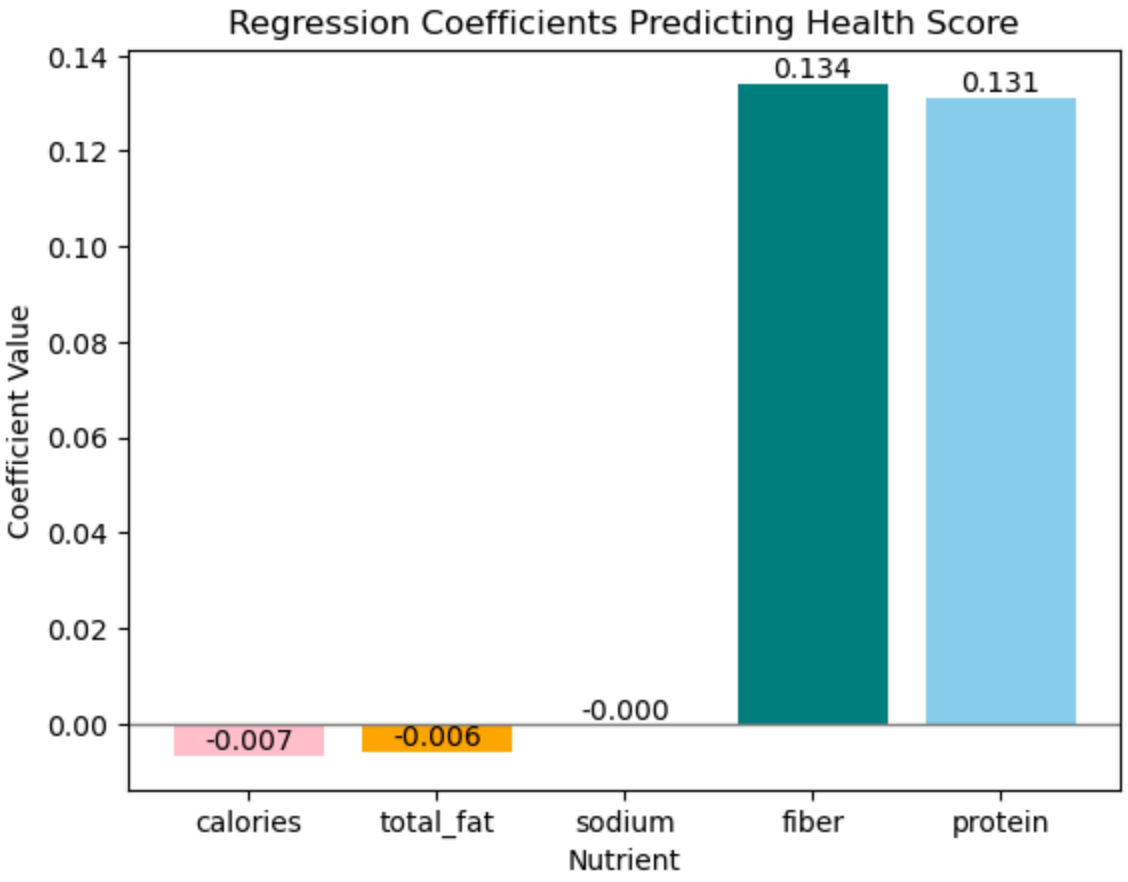
```
/opt/anaconda3/lib/python3.13/site-packages/scipy/stats/_axis_nan_policy.py:
430: UserWarning: `kurtosistest` p-value may be inaccurate with fewer than 2
0 observations; only n=9 observations were given.
  return hypotest_fun_in(*args, **kwds)
```

In [391…

```
coef = model.params.drop("Intercept")
coef_df = pd.DataFrame({
    "nutrient": coef.index,
    "coefficient": coef.values})

plt.figure
bars = plt.bar(coef_df["nutrient"], coef_df["coefficient"], color=["pink", "

for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, height, f"{height:.3f}",
             ha="center", va="bottom")
plt.title("Regression Coefficients Predicting Health Score")
plt.ylabel("Coefficient Value")
plt.xlabel("Nutrient")
plt.axhline(0, color='gray', linewidth=1)
plt.show()
```



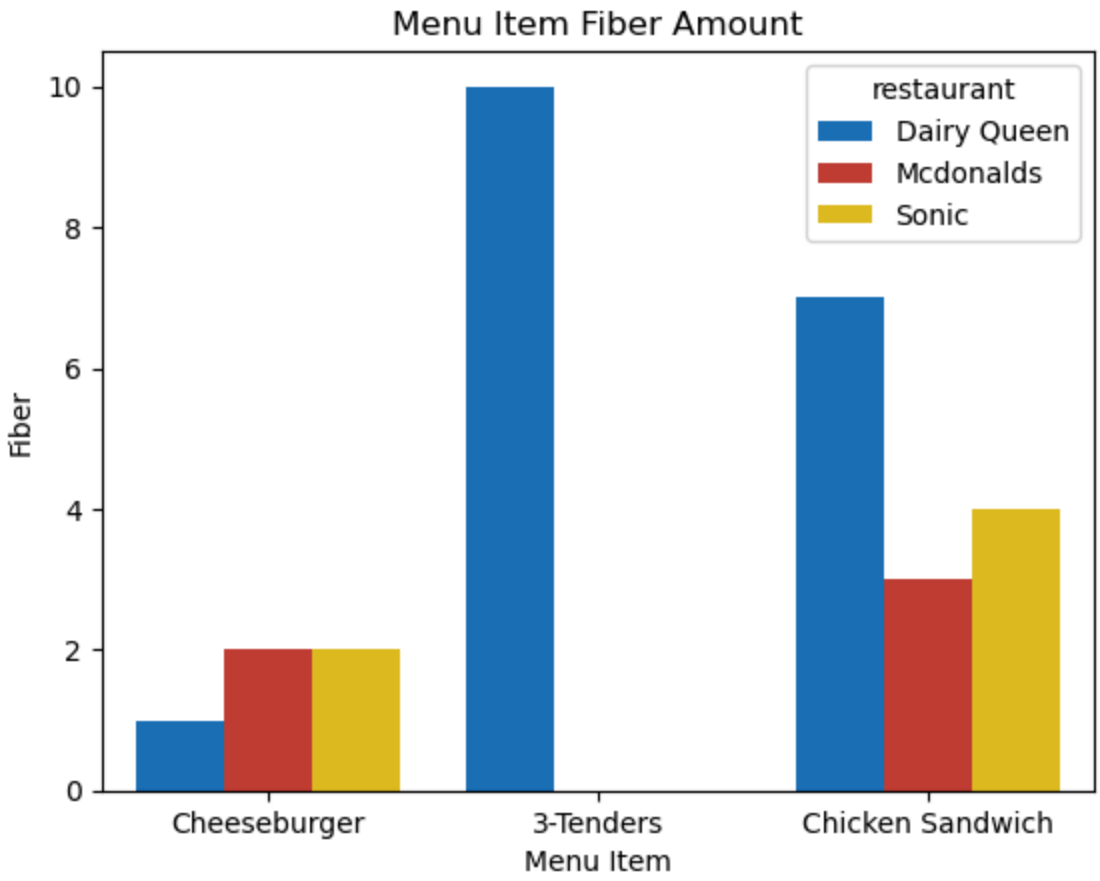
In [379…

```
sns.barplot(
    data=df,
    x='menu_item',
    y='fiber',
    hue='restaurant',
```

```
    palette=['#0072CE', '#DA291C', '#FFD100'] )


plt.title('Menu Item Fiber Amount')
plt.xlabel('Menu Item')
plt.ylabel('Fiber')

plt.show()
```

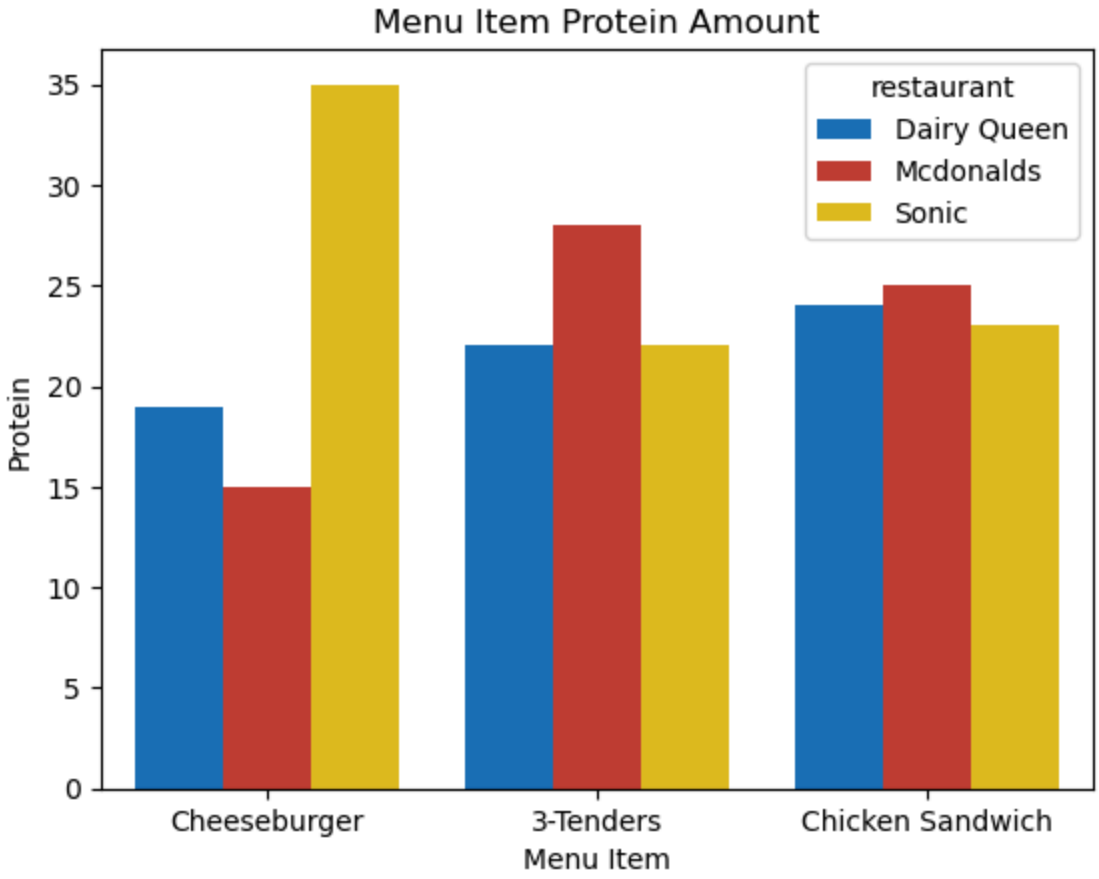


In [380…

```
sns.barplot(
    data=df,
    x='menu_item',
    y='protein',
    hue='restaurant',
    palette=['#0072CE', '#DA291C', '#FFD100'] )


plt.title('Menu Item Protein Amount')
plt.xlabel('Menu Item')
plt.ylabel('Protein')

plt.show()
```


Menu Item Protein Amount
restaurant
Dairy Queen
Mcdonalds
Sonic
Protein
0
5
10
15
20
25
30
35
Cheeseburger
3-Tenders
Chicken Sandwich
Menu Item